

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA ĐIỆN TỬ - VIỄN THÔNG



BÁO CÁO NHẬP MÔN HỆ THỐNG NHÚNG

THIẾT KẾ XE TỰ HÀNH SỬ DỤNG STM32
KẾT HỢP GIẢI THUẬT PID
VÀ XÂY DỰNG BẢN ĐỒ SA HÌNH THỜI GIAN THỰC

Giảng viên hướng dẫn : PGS.TS Phạm Đức Quang

Mã lớp học : ELT_3240_2

Thành viên nhóm : Đặng Anh Tài – 23021900
: Đặng Trung Thành – 23021918
: Trần Minh Thông – 23021922
: Trần Anh Tuấn – 23021910

MỤC LỤC

MỤC LỤC	1
DANH MỤC HÌNH ẢNH.....	2
DANH MỤC BẢNG BIỂU.....	2
CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN.....	3
1.1. Yêu cầu dự án và mục tiêu thiết kế.....	3
1.1.1. Yêu cầu dự án.....	3
1.1.2. Mục tiêu thiết kế.....	3
1.2. Thành viên nhóm và Bảng phân chia đóng góp nội dung	4
CHƯƠNG 2: ĐẶC TẢ HỆ THỐNG PHẦN CỨNG	4
2.1. Sơ đồ khối tổng thể của hệ thống	4
2.2. Danh sách linh kiện sử dụng.....	5
CHƯƠNG 3: THIẾT KẾ PHẦN MỀM	6
3.1. Phân bổ tài nguyên và định nghĩa chân tín hiệu.....	6
3.2. Lưu đồ thuật toán tổng thể.....	8
3.3. Thuật toán điều khiển PID rời rạc	9
3.3.1. Công thức toán học PID rời rạc ứng dụng cho vi điều khiển.....	9
3.3.2. Giải thuật tính toán sai số vị trí	10
3.3.3. Quy trình tinh chỉnh hệ số (Tuning).....	12
3.4. Logic xử lý sa hình nâng cao.....	13
3.4.1. Bộ lọc nhiễu động	13
3.4.2. Thuật toán Coast & Scan.....	13
3.4.3. Cơ chế khóa ngã rẽ.....	14
3.5. Giao thức truyền thông UART và dựng sa hình	14
3.5.1. Định dạng gói tin truyền thông	14
3.5.2. Giải thuật định vị không gian và trực quan hóa	15

CHƯƠNG 4: KHẢO SÁT VÀ KẾT QUẢ	16
4.1. Kết quả vận hành phần cứng của xe trên sa hình thực tế	16
4.2. Kết quả trực quan hóa bản đồ trên phần mềm máy tính.....	16
4.3. Sai số và Phân tích sai số.....	18
4.3.1. Các sai số thực nghiệm ghi nhận được trong quá trình lập bản đồ.....	18
4.3.2. Phân tích nguyên nhân gây ra sai số	19
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	20
5.1. Kết luận.....	20
5.2. Hướng phát triển.....	20
TÀI LIỆU THAM KHẢO	21

DANH MỤC HÌNH ẢNH

Hình 1. Sơ đồ khối tổng thể của hệ thống	4
Hình 2. Danh sách port của NUCLEO-F401RE	7
Hình 3. Pinout của module driver động cơ TB6612FNG	7
Hình 4. Pinout của module UART bluetooth JDY-31-SPP	7
Hình 5. Lưu đồ thuật toán của xe tự hành.....	8
Hình 6. Sa hình dựng lại trên máy tính	17
Hình 7. Sa hình thực tế.....	17
Hình 8. Sa hình thiếu các ngã rẽ do thiết lập thời gian chờ quá cao	19
Hình 9. Sa hình quá nhiều ngã rẽ do thiết lập thời gian chờ quá ngắn	19

DANH MỤC BẢNG BIỂU

Bảng 1. Bảng phân chia công việc thành viên trong nhóm.....	4
Bảng 2. Danh sách linh kiện sử dụng.....	5
Bảng 3. Bảng quy hoạch chân tín hiệu giao tiếp phần cứng	6
Bảng 4. Trọng số của các cảm biến	11
Bảng 5. Sai số giữa sa hình thực tế và sa hình dựng lại bằng phần mềm	18

CHƯƠNG 1: TỔNG QUAN VỀ DỰ ÁN

1.1. Yêu cầu dự án và mục tiêu thiết kế

1.1.1. Yêu cầu dự án

Dự án đặt ra bài toán thiết kế và chế tạo một hệ thống xe tự hành kích thước nhỏ có khả năng tự động bám theo quỹ đạo vạch kẻ sẵn và đồng thời truyền dữ liệu hành trình về máy trạm để vẽ lại bản đồ không gian. Các yêu cầu kỹ thuật cụ thể bao gồm:

- Xe phải di chuyển ổn định trên vạch dẫn đường cơ bản và xử lý được các đặc tuyến hình học cũng như vạch đích.
- Mạch điều khiển phải tính toán được thời gian tịnh tiến, nhận diện các mốc bề lái và đóng gói dữ liệu để gửi về máy tính thông qua giao thức truyền thông không đồng bộ theo thời gian thực. Từ đó, xây dựng phần mềm trên máy tính tiếp nhận dữ liệu từ xe, tiến hành nội suy hình học để vẽ lại hình dáng sa hình thực tế trên giao diện đồ họa.

1.1.2. Mục tiêu thiết kế

Về phần cứng, nhóm lựa chọn cấu hình động học vi sai với 2 bánh chủ động và bánh trước điều hướng, kết hợp mảng cảm biến hồng ngoại 5 mắt bố trí phía trước mũi xe để tăng độ phân giải khi quét vạch. Xe sử dụng vi điều khiển kiến trúc ARM Cortex-M4 là STM32 làm lõi xử lý trung tâm, đảm bảo xung nhịp cao để đáp ứng tần số lấy mẫu tín hiệu và xuất xung PWM điều khiển động cơ DC giảm tốc.

Về phần mềm nhúng, nhóm áp dụng phương pháp lập trình trực tiếp trên thanh ghi, triển khai bộ điều khiển PID rời rạc để điều chỉnh độ lệch tâm, kết hợp với các bộ đếm thời gian thực để xây dựng bộ lọc nhiễu cơ học và khóa an toàn tại các ngã rẽ.

Về phần mềm máy tính, nhóm phát triển trên ngôn ngữ Python, sử dụng thư viện Matplotlib để trực quan hóa dữ liệu, triển khai giải thuật ước lượng vị trí để chuyển đổi từ dữ liệu thời gian chạy và hướng rẽ nhận từ vi điều khiển thành các vector không gian, từ đó dựng lên bản đồ hành trình thực tế của robot.

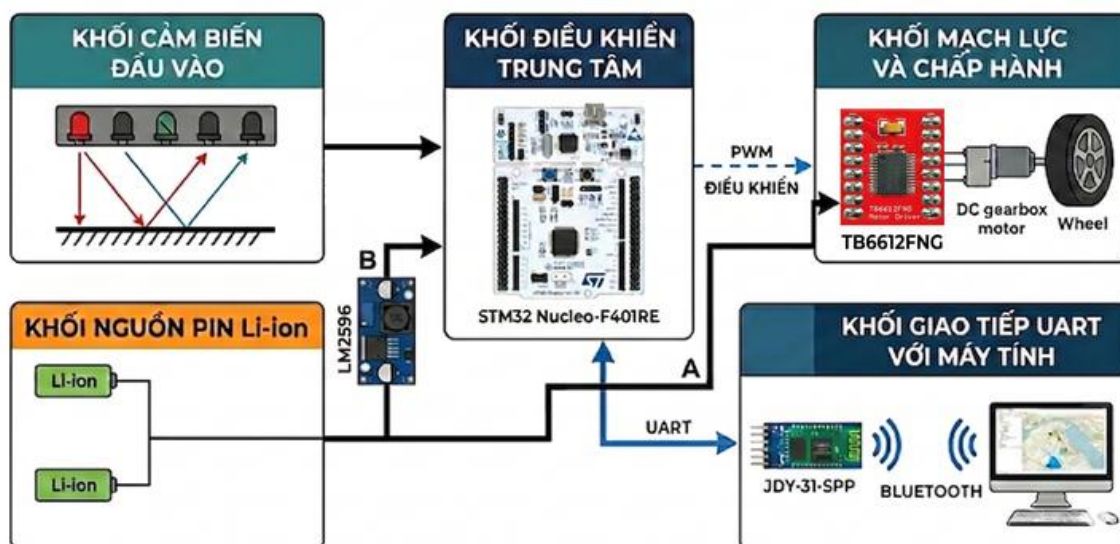
1.2. Thành viên nhóm và Bảng phân chia đóng góp nội dung

Bảng 1. Bảng phân chia công việc thành viên trong nhóm

STT	Họ và tên	Mã sinh viên	Đóng góp
1	Đặng Anh Tài	23021900	Cấu hình thanh ghi phần cứng, mạch công suất và hàm điều khiển động cơ
2	Trần Minh Thông	23021922	Xử lý tín hiệu cảm biến và viết thuật toán PID bám line thẳng
3	Đặng Trung Thành	23021918	Xử lý logic chiến thuật bề lái ,thuật toán lọc nhiễu và xử lý sa hình
4	Trần Anh Tuấn	23021910	Chịu trách nhiệm thiết lập giao thức UART, xây dựng phần mềm máy tính để trực quan hóa đồ họa bản đồ

CHƯƠNG 2: ĐẶC TẢ HỆ THỐNG PHẦN CỨNG

2.1. Sơ đồ khối tổng thể của hệ thống



Hình 1. Sơ đồ khối tổng thể của hệ thống

Hệ thống xe tự hành dò line được thiết kế theo mô hình điều khiển tập trung. Luồng tín hiệu hoạt động như sau: khối cảm biến quang học quét liên tục bề mặt sa hình để thu thập trạng thái vạch dẫn, truyền tín hiệu số về khối xử lý trung tâm. Dựa trên dữ liệu này, vi điều khiển tính toán thuật toán PID và xuất tín hiệu điều chế độ rộng xung PWM đến khối chấp hành để điều áp cho hai động cơ DC, giúp xe bám quỹ đạo. Đồng thời, các trạng thái thời gian và góc cua được đóng gói gửi qua giao thức UART về máy tính để vẽ bản đồ.

2.2. Danh sách linh kiện sử dụng

Bảng 2. Danh sách linh kiện sử dụng

STT	Tên linh kiện	Số lượng	Chức năng
1	Mạch phát triển STM32 NUCLEO-F401RE	1	Vi điều khiển trung tâm
2	N20, điện áp 6V, tốc độ 600 RPM	2	Động cơ DC
3	Driver cầu H TB6612FNG	1	Điều khiển động cơ DC
4	TCRT5000 với mảng 5 mắt thu phát	1	Module cảm biến hồng ngoại
5	LM2596	1	Module giảm áp
6	Bánh trước tự do	1	Giữ cân bằng
7	Bánh sau cao su	2	Truyền động tịnh tiến và rẽ
8	Pin Li-ion	2	Cấp nguồn
9	Breadboard và dây cắm	1	Kết nối các linh kiện
10	Tụ gốm 104	2	Lọc nhiễu động cơ
11	Khung xe mica	1	Giá đỡ hệ thống
12	JDY-31-SPP	1	Module UART-USB
13	Công tắc	1	Bật/tắt

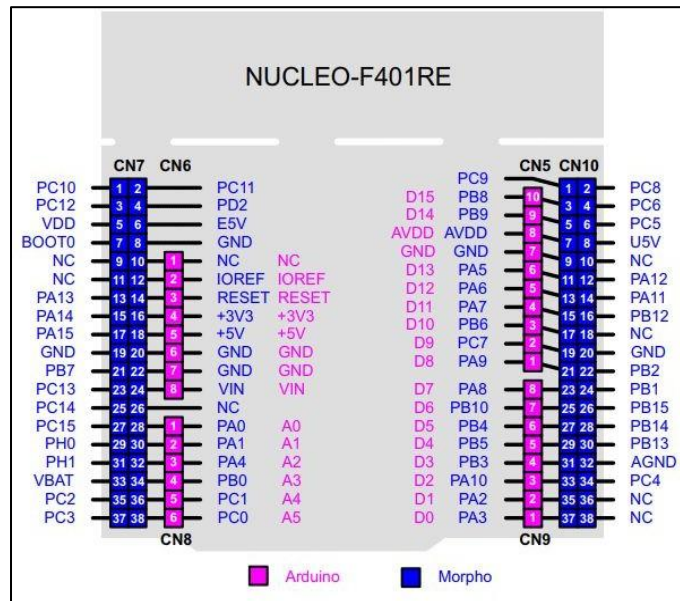
CHƯƠNG 3: THIẾT KẾ PHẦN MỀM

3.1. Phân bổ tài nguyên và định nghĩa chân tín hiệu

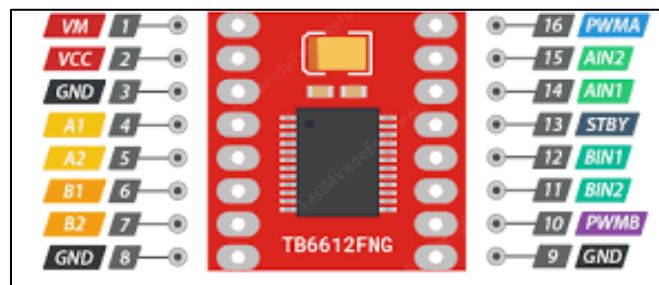
Để đảm bảo khả năng đáp ứng thời gian thực và quản lý tài nguyên vi điều khiển hiệu quả, việc phân bổ các chân GPIO được thiết kế tối ưu theo từng cụm ngoại vi.

Bảng 3. Bảng quy hoạch chân tín hiệu giao tiếp phần cứng

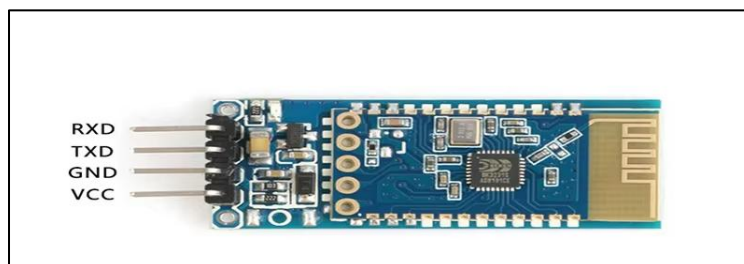
Module	Tên tín hiệu	Port STM32
Cảm biến hồng ngoại TCRT5000	Mắt L2 (Trái ngoài)	PA0
	Mắt L1 (Trái trong)	PA1
	Mắt C (Trung tâm)	PA4
	Mắt R1 (Phải trong)	PB0
	Mắt R2 (Phải ngoài)	PC1
	Nguồn Cảm biến	5V / GND
Driver Động cơ TB6612FNG	AIN1 (Chiều quay Trái 1)	PC7
	AIN2 (Chiều quay Trái 2)	PA9
	PWMA (Tốc độ Trái)	PB6
	BIN1 (Chiều quay Phải 1)	PA8
	BIN2 (Chiều quay Phải 2)	PB10
	PWMB (Tốc độ Phải)	PB7
	STBY (Standby)	PB5
	Nguồn Động lực (VM)	Không (nối với Pin)
	Nguồn Logic (VCC)	3.3V / GND
Module Bluetooth JDY-31-SPP	TX (Truyền dữ liệu)	PA12
	RX (Nhận lệnh)	PC6
	Nguồn Bluetooth	5V / GND



Hình 2. Danh sách port của NUCLEO-F401RE



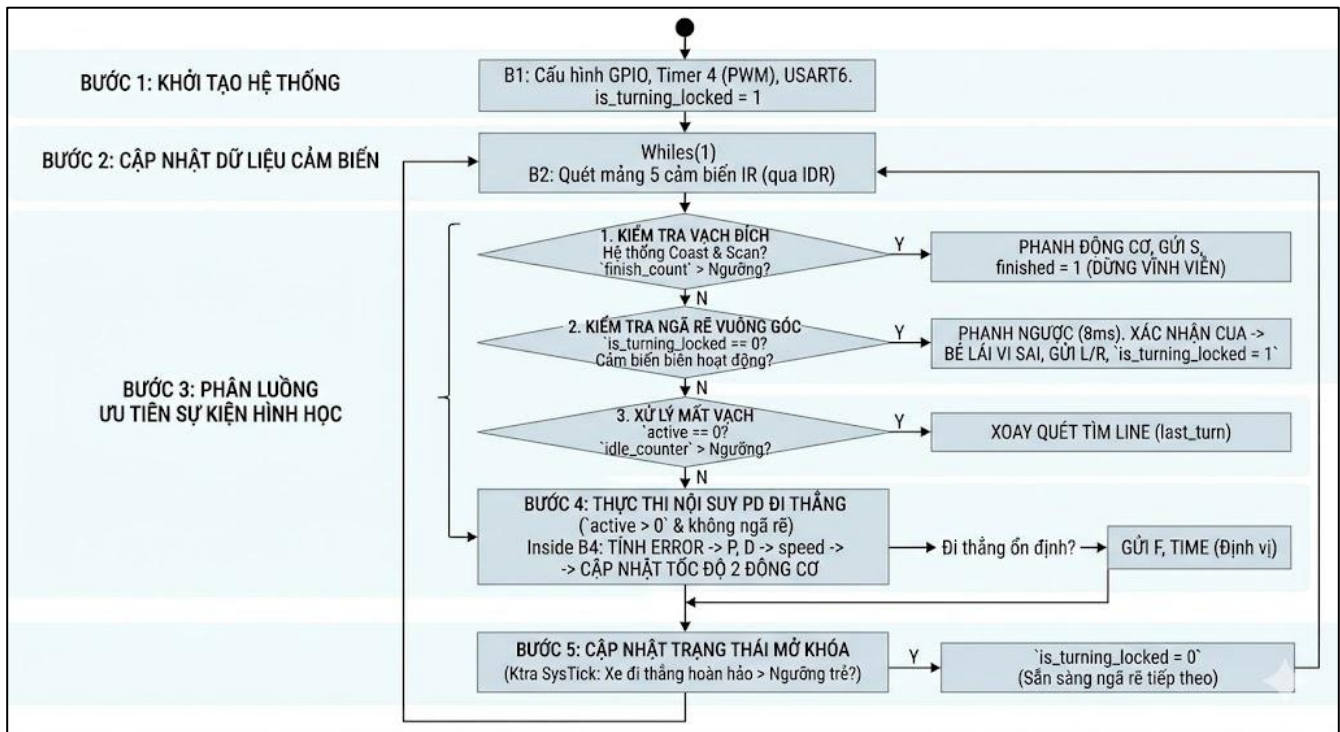
Hình 3. Pinout của module driver động cơ TB6612FNG



Hình 4. Pinout của module UART bluetooth JDY-31-SPP

Chương trình của xe tự hành và mô phỏng sa hình : [link](#)

3.2. Lưu đồ thuật toán tổng thể



Hình 5. Lưu đồ thuật toán của xe tự hành

Chương trình điều khiển phần mềm của vi điều khiển STM32 được xây dựng theo kiến trúc vòng lặp tuần tự liên tục kết hợp với các bộ định thời thời gian thực - SysTickTimer. Nguyên lý cốt lõi của lưu đồ là ưu tiên phân luồng xử lý các sự kiện đột biến hình học trước khi cấp quyền cho hàm nội suy PD đi thẳng. Kiến trúc này giúp hệ thống đạt được thời gian đáp tối ưu nhất đối với các biến thiên của sa hình.

Quy trình vận hành tổng thể được chia thành các giai đoạn chính như sau:

- Bước 1 của lưu đồ là khởi tạo hệ thống. Ngay khi cấp nguồn, vi điều khiển thực hiện cấu hình các thanh ghi ngoại vi bao gồm: thiết lập chân GPIO, cấu hình Timer 4 để xuất xung PWM điều khiển động cơ, và khởi tạo chuẩn giao tiếp USART6. Biến khóa ngã rẽ `is_turning_locked` được khởi tạo bằng 1 để đưa xe vào trạng thái ưu tiên đi thẳng.
- Bước 2 là cập nhật dữ liệu cảm biến. Tại đầu mỗi chu kỳ của vòng lặp `while(1)`, vi điều khiển quét trạng thái nhị phân của mảng 5 cảm biến hồng ngoại thông qua thanh ghi Input Data Register (IDR). Dữ liệu này là cơ sở để hệ thống quyết định rẽ nhánh logic ở các bước tiếp theo.
- Bước 3 là phân luồng ưu tiên sự kiện hình học và ra quyết định trạng thái. Đầu tiên thuật toán thực hiện kiểm tra vạch đích. Hệ thống quét điều kiện thuật toán *Coast & Scan*. Nếu

thỏa mãn điều kiện vạch ngang và vượt quá ngưỡng đếm `finish_count`, xe lập tức phanh động cơ, gửi ký tự S qua UART và rơi vào trạng thái dừng vĩnh viễn (`finished = 1`). Tiếp theo, thuật toán kiểm tra ngã rẽ vuông góc. Nếu không phải vạch đích, hệ thống kiểm tra trạng thái các cảm biến biên (L2, L1 hoặc R2, R1). Khối lệnh này chỉ được cấp phép thực thi khi cờ `is_turning_locked` đã được mở. Nếu phát hiện ngã rẽ, hệ thống kích hoạt bộ lọc nhiễu động (phanh ngược 8ms). Nếu xác nhận đúng là góc cua, xe thực hiện bẻ lái vì sai, phát gói tin L hoặc R qua UART, thiết lập lại mốc thời gian và lập tức đóng cờ `is_turning_locked = 1`. Cuối cùng là xử lý ngoại lệ mất vạch. Nếu toàn bộ cảm biến báo vùng trắng (`active == 0`), hệ thống kích hoạt biến đếm mất vạch `idle_counter`. Vượt qua ngưỡng cho phép, xe tự động thực thi chu trình xoay quét tìm line dựa trên biến nhớ hướng rẽ gần nhất (`last_turn`).

- Bước 4 là thực thi nội suy PD đi thẳng. Nếu tín hiệu cảm biến không rơi vào ba trường hợp sự kiện ở Bước 3, hệ thống sẽ tiến hành chạy thuật toán bám quỹ đạo: tính toán sai số vị trí Error dựa trên phương pháp trọng số -> Cập nhật khâu Tỷ lệ P và Vi phân D để xuất ra giá trị hiệu chỉnh speed -> Cộng/trừ đối xứng biến speed vào tốc độ nền `basespeed` của hai động cơ trái và phải, duy trì xe ở trạng thái cân bằng động. Nếu xe đang đi thẳng ổn định, hệ thống đóng gói thời gian chạy và phát bản tin F, time lên máy tính để phục vụ thuật toán định vị không gian.
- Bước 5: Cập nhật trạng thái mở khóa. Ở cuối chu kỳ lặp, hệ thống kiểm tra bộ định thời SysTick. Nếu xe đã duy trì trạng thái đi thẳng hoàn hảo (mất trung tâm C đè vạch) vượt quá ngưỡng thời gian trễ chống nhiễu thiết lập trước, vi điều khiển sẽ giải phóng cờ khóa ngã rẽ (`is_turning_locked = 0`). Thao tác này đưa hệ thống về trạng thái sẵn sàng nhận diện các giao lộ hoặc góc cua tiếp theo, khép kín một vòng lặp tuần tự.

3.3. Thuật toán điều khiển PID rời rạc

3.3.1. Công thức toán học PID rời rạc ứng dụng cho vi điều khiển.

Trong các hệ thống robot dò line hoạt động thời gian thực, bộ điều khiển PID (Proportional – Integral – Derivative) được sử dụng nhằm tạo tín hiệu hiệu chỉnh tốc độ vi sai giữa hai động cơ, từ đó duy trì tâm xe bám sát đường dẫn mong muốn. Do hệ thống chạy trên vi điều khiển STM32 và thực hiện tính toán theo từng chu kỳ rời rạc trong vòng lặp `while(1)`, mô hình PID liên tục cần được chuyển đổi sang dạng PID rời rạc.

Biểu thức tổng quát của bộ điều khiển PID liên tục được mô tả như sau:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

Trong đó:

- $e(t)$: sai số vị trí giữa tâm xe và vạch line.
- K_p : hệ số tỉ lệ, quyết định tốc độ phản ứng tức thời.
- K_i : hệ số tích phân, dùng triệt tiêu sai lệch tĩnh.
- K_d : hệ số vi phân, làm giảm dao động và độ vọt lố.

Khi triển khai trên STM32, thành phần tích phân được loại bỏ và hệ điều khiển được rút gọn thành bộ điều khiển PD. Công thức thực thi trong chương trình được biểu diễn:

$$P = e(k)$$

$$D = e(k) - e(k-1)$$

$$u(k) = K_p P + K_d D$$

Giá trị điều khiển speed được cộng và trừ đối xứng vào tốc độ nền của hai bánh xe:

$$V_L = V_{base} - u(k)$$

$$V_R = V_{base} + u(k)$$

Trong đó:

- V_L : tốc độ động cơ trái.
- V_R : tốc độ động cơ phải.
- V_{base} : tốc độ cơ bản.

Nếu sai số dương ($error > 0$), xe lệch về bên phải nên tốc độ bánh trái giảm và bánh phải tăng nhằm kéo xe trở về tâm line. Ngược lại, khi sai số âm, xe sẽ quay hiệu chỉnh về phía trái.

Để bảo vệ phần cứng và tránh hiện tượng bão hòa điều khiển (Saturation), tốc độ hai bánh được giới hạn trong khoảng: $-100 < V < 100$.

Việc chặn biên này giúp động cơ không vượt quá Duty Cycle cho phép của PWM và giữ ổn định cho mạch.

3.3.2. Giải thuật tính toán sai số vị trí

Một trong những bài toán quan trọng nhất của xe dò line là xác định vị trí tương đối giữa tâm xe và đường dẫn. Hệ thống sử dụng mảng cảm biến hồng ngoại gồm 5 mắt được bố trí tuyến tính theo thứ tự:

L2-L1-C-R1-R2

Trong đó: L2, L1, C, R1, R2 lần lượt là các cảm biến từ ngoài cùng bên trái đến ngoài cùng bên phải.

Để chuyển đổi trạng thái logic thành tọa độ liên tục phục vụ PID, nhóm áp dụng phương pháp nội suy trọng số (Weighted Position Interpolation). Mỗi mắt cảm biến được gán một trọng số cố định:

Bảng 4. Trọng số của các cảm biến

Cảm biến	L2	L1	C	R1	R2
Trọng số	1000	1500	2000	2500	3000

Trong chương trình: `int weight[] = {1000, 1500, 2000, 2500, 3000};`

Nếu có n cảm biến đang phát hiện line, vị trí trung bình được xác định:

$$Position = \frac{\sum_{i=1}^n S_i W_i}{\sum_{i=1}^n S_i}$$

Trong đó:

- S_i : trạng thái cảm biến (0 hoặc 1).
- W_i : trọng số tương ứng.

Ví dụ:

- $C = 1 \Rightarrow Position = 2000 \Rightarrow Error = 2000 - 2000 = 0$

Xe sẽ đi thẳng.

- $L1 = 1, C = 1 \Rightarrow Position = (1500 + 2000)/2 = 1750 \Rightarrow Error = 1750 - 2000 = -250$

Bộ điều khiển hiểu rằng xe lệch trái và tạo lệnh quay phải.

- $R1 = 1, R2 = 1 \Rightarrow Position = (2500 + 3000)/2 = 2750 \Rightarrow Error = 2750 - 2000 = 750$

Xe sẽ được điều chỉnh quay trái.

Ưu điểm của phương pháp nội suy trọng số là tạo được biến vị trí liên tục thay vì tín hiệu nhị phân ON/OFF đơn thuần. Nhờ đó bộ điều khiển PID nhận được sai số mượt hơn, hạn chế hiện tượng rung giật khi xe di chuyển trên các đoạn cong.

Khi toàn bộ cảm biến mất tín hiệu: $active = 0$

Hệ thống chuyển sang chế độ xử lý mất line thông qua biến `idle_counter`, cho phép xe tiếp tục trôi quán tính hoặc quét tìm lại đường dựa trên hướng lưu bởi `last_turn`.

3.3.3. Quy trình tinh chỉnh hệ số (Tuning)

Quá trình xác định các thông số PID không được thực hiện bằng phương pháp giải tích mà sử dụng kỹ thuật tinh chỉnh thực nghiệm trực tiếp trên sa hình thật.

Ban đầu toàn bộ hệ số được đặt: $K_p=K_d=K_i=0$, $basespeed = 20$ (tốc độ cơ bản thấp để đánh giá các hành vi cơ học).

Giai đoạn thứ nhất tiến hành tăng dần hệ số tỉ lệ K_p . Khi giá trị quá nhỏ, xe phản ứng chậm, thường bị lệch khỏi tâm line trước khi kịp hiệu chỉnh. Khi tăng vượt ngưỡng, xe xuất hiện dao động hai bên và sinh hiện tượng “lắc đuôi”. Qua các lần thử nghiệm $K_p = 0.035$ là giá trị tối ưu nhất, đủ để tạo phản ứng nhanh nhưng không gây dao động quá mạnh.

Giai đoạn tiếp theo bổ sung thành phần vi phân. Khâu vi phân hoạt động như một bộ giảm chấn (Damping Element), dự đoán xu hướng thay đổi sai số: $D = e(k)-e(k-1)$. Nếu xe bắt đầu lệch nhanh khỏi quỹ đạo, thành phần D tăng mạnh và tạo lực hãm ngược chiều, làm giảm độ vọt quá đà. $K_d=0.3$ là giá trị tốt nhất để thực hiện chức năng trên.

Sau thực nghiệm, nhóm nhận thấy việc đưa thêm tích phân gây tác động bất lợi. Thành phần: $I=e(k)$ làm sai số tích lũy liên tục trong các đoạn cua gắt. Khi xe thoát khỏi khúc cua, giá trị tích phân vẫn còn tồn tại, tạo hiện tượng vượt tâm line, dao động kéo dài, tăng thời gian ổn định, sinh rung mạnh tại góc cua.

Do đặc tính hệ dò line có sai số thay đổi liên tục và không tồn tại sai lệch tĩnh kéo dài, nhóm quyết định để: $K_i = 0$ và chuyển sang bộ điều khiển PD.

Với phần tốc độ cơ bản: $basespeed = 40$ cũng được xác định thực nghiệm. Nếu giá trị quá thấp, xe di chuyển ổn định nhưng thời gian hoàn thành sa hình lớn. Khi tăng vượt ngưỡng, quán tính cơ học xuất hiện mạnh làm giảm hiệu quả của khâu vi phân gây ra hiện tượng vượt quá qua các đoạn cua gắt và tốn nhiều thời gian để lấy lại line hơn.

Kết quả thực nghiệm cho thấy bộ thông số $K_p=0.035$, $K_d=0.3$, $K_i=0$, $basespeed = 40$ cho khả năng bám line ổn định, giảm dao động thân xe, xử lý tốt các góc cua vuông và duy trì độ chính xác đủ cao cho quá trình lập bản đồ thời gian thực trên máy tính.

3.4. Logic xử lý sa hình nâng cao

3.4.1. Bộ lọc nhiễu động

Trong quá trình di chuyển tốc độ cao, bộ điều khiển PID sẽ liên tục tạo ra các xung gia tốc để điều chỉnh hướng. Quán tính này thường gây ra hiện tượng lắc, khiến cho các mắt cảm biến rìa (L2 và R2) vô tình quét qua vạch kẻ trong thời gian rất ngắn. Nếu hệ thống phản ứng ngay lập tức với tín hiệu nhanh, xe sẽ nhận diện nhầm đây là một góc cua trên đường thẳng và bề lái sai lệch.

Từ đó, nhóm áp dụng toán lọc nhiễu động dựa trên cơ chế delay kết hợp với vòng lặp kín

Trong quá trình xử lý khúc cua 90 độ, ngay khi phát hiện trạng thái rẽ trái hợp lệ, hệ thống không bề lái ngay mà thực hiện lệnh hãm động cơ: `motor_control(-30, -30); delay_ms(8)`.

Điều này tạo ra một khoảng trễ nhằm triệt tiêu quán tính tiến và chờ tín hiệu từ cảm biến ổn định trở lại. Sau khoảng trễ, hệ thống sẽ thực hiện một bước kiểm tra trạng thái của cảm biến: `if(!(GPIOC->IDR & (1<<1)))` (đối với rẽ trái). Nếu đây chỉ là một xung nhiễu, cảm biến sẽ không còn ở trên vạch nữa, hệ thống sẽ thoát khỏi lệnh rẽ và tiếp tục đi thẳng. Nếu tín hiệu vẫn giữ nguyên, hệ thống sẽ thoát khỏi lệnh rẽ và tiếp tục đi thẳng. Nếu tín hiệu vẫn giữ nguyên, hệ thống mới chính thức xác nhận đây là góc cua thật và thực thi vòng lặp xoay trục. Cơ chế này hoạt động tương đương với một bộ lọc thông thấp (Low-pass filter) trong miền thời gian, giúp loại bỏ các gai nhiễu tần số cao.

3.4.2. Thuật toán Coast & Scan

Khi xe tiến vào vạch đích từ một góc chéo do ảnh hưởng của sai số PID trước đó, xe có thể gặp tình trạng “mù” tạm thời. Hiện tượng này xảy ra khi các cảm biến rìa chạm vạch đích trước nhưng do xe nằm chéo và cảm biến đối diện không kích hoạt, khiến hệ thống không thỏa mãn điều kiện `L2 && R2 && (L1 || C || R1)` để nhận diện vạch đích.

Nhóm xây dựng kỹ thuật tịnh tiến động cơ phổ hợp khung thời gian (Coast & Scan) thông qua biến đếm `finish_count`. Khi hệ thống phát hiện một tín hiệu có khả năng là vạch đích thay vì dừng lại ngay lập tức, bộ đếm `finish_count++` được tăng lên. Xe tiếp tục duy trì tốc độ tiến thẳng nhờ các lệnh PID vẫn đang được chạy ngầm, tạo cơ hội cho toàn bộ dàn cảm biến trượt qua vạch ngang.

Hệ thống liên tục scan trong các chu kỳ lặp tiếp theo. Chỉ khi điều kiện vạch đích duy trì liên tục và vượt qua ngưỡng `finish_count > 4` (tương đương khoảng 40 - 50ms trượt trên vạch), biến

finished mới được lên set 1. Nếu xe trượt ra khỏi vạch trước khi đạt ngưỡng do nhận nhầm một mảng chéo finish_count sẽ bị kéo về 0 giúp xe thoát khỏi trạng thái “ mù “ và tiếp tục bám line.

3.4.3. Cơ chế khóa ngã rẽ

Các sa hình thi đấu thường có những đoạn đường hẹp nối tiếp các khúc cua liên tục (Z-shape). Sau khi thoát khỏi một góc cua, thân xe chưa kịp thẳng lại, việc kích hoạt ngay lập tức bộ nhận diện ngã rẽ có thể gây ra hiện tượng trùng lặp lệnh (Double-triggering), làm xe xoay vòng liên tục tại một điểm.

Để giải quyết, nhóm triển khai một cờ trạng thái (State Flag) kết hợp với bộ định thời (Timer) dựa trên trạng thái ổn định của cảm biến trung tâm.

Biến is_turning_locked được khởi tạo bằng 1 (Khóa) ngay khi cấp nguồn và ngay sau khi hoàn thành bất kỳ góc cua nào. Điều này cách ly hoàn toàn khối lệnh xử lý rẽ khỏi hệ thống điều khiển chính. Hệ thống liên tục theo dõi trạng thái C && !L2 && !R2 (chỉ duy nhất cảm biến trung tâm nhận line). Khi điều kiện này xảy ra, biến clear_line_timer ghi nhận mốc thời gian get_tick().

Bộ nhận diện ngã rẽ chỉ được cấp phép hoạt động trở lại (Unlock: is_turning_locked = 0) khi và chỉ khi trạng thái đi thẳng hoàn hảo này được duy trì liên tục vượt quá chu kỳ x ms (get_tick() - clear_line_timer > x). Cơ chế này tạo ra một "vùng cấm nhận diện" ngay sau mỗi khúc cua, đảm bảo xe phải di chuyển ổn định trên một đoạn đường thẳng nhất định trước khi được phép phân tích giao lộ tiếp theo, ngăn ngừa hoàn toàn hiện tượng loạn trạng thái.

3.5. Giao thức truyền thông UART và dựng sa hình

3.5.1. Định dạng gói tin truyền thông

Hệ thống không truyền liên tục toàn bộ tọa độ do giới hạn về băng thông và tốc độ xử lý của vi điều khiển, mà sử dụng cơ chế truyền "sự kiện". Khi xe kết thúc một hành động động học, mạch STM32 sẽ đóng gói và phát một bản tin chứa ký tự mã hóa qua chuẩn USART6 (Baudrate 9600) lên máy tính.

Khi xe vừa hoàn thành một đoạn đường thẳng và bắt đầu bẻ lái, STM32 sẽ tính toán khoảng thời gian đi thẳng (tính bằng mili-giây thông qua hàm get_tick()) và gửi chuỗi ký tự định dạng F,xxxx\n (ví dụ: F,1250\n báo hiệu xe đã chạy thẳng liên tục 1250 ms). Gói tin rẽ trái (L\n) được gửi ngay sau khi hàm kiểm tra điều kiện rẽ trái hoàn tất và vòng lặp rẽ vi sai được thực thi. Gói tin rẽ phải (R\n) được gửi ngay sau khi hàm kiểm tra điều kiện rẽ phải hoàn tất. Gói tin vạch đích

(S\n) được gửi khi bộ đếm finish_count xác nhận xe đã quét qua vạch đích, yêu cầu máy tính chốt điểm cuối của bản đồ.

3.5.2. Giải thuật định vị không gian và trực quan hóa

Do cấu hình phần cứng không sử dụng cảm biến đo góc quay hay vận tốc (Encoder), phần mềm nội suy tọa độ hoàn toàn dựa trên giải thuật ước lượng vị trí với các bước sau:

- Hệ thống bắt đầu vẽ từ gốc tọa độ (0, 0) và giả định hướng ban đầu của mũi xe trở thẳng đứng lên trên theo trục Y, tương ứng với góc xoay ban đầu:

$$\theta = 90^\circ$$

- Khi nhận được bản tin đi thẳng cùng tham số thời gian t, phần mềm quy đổi ra khoảng cách đồ họa D thông qua hệ số tỷ lệ SCALE:

$$D = t \times SCALE$$

Tọa độ điểm nối tiếp theo được tính toán thông qua phép chiếu lượng giác dựa trên tọa độ điểm cũ và góc xoay hiện tại:

$$x_{new} = x_{old} + D \cdot \cos(\theta)$$

$$y_{new} = y_{old} + D \cdot \sin(\theta)$$

- Khi nhận được lệnh rẽ trái, hệ thống không làm thay đổi tọa độ tịnh tiến mà chỉ thực hiện phép cộng/trừ góc để định hướng cho phân đoạn vẽ tiếp theo. Rẽ trái làm mũi xe quay ngược chiều kim đồng hồ nên góc xoay tăng 90 độ. Rẽ phải làm mũi xe quay cùng chiều kim đồng hồ nên góc xoay giảm 90 độ.
- Khi nhận bản tin S, phần mềm dừng cập nhật quỹ đạo, đánh dấu mốc tọa độ cuối cùng bằng điểm đánh dấu (Marker) màu xanh lá và hiển thị thông báo hoàn thành sa hình, kết thúc quá trình dựng bản đồ.

CHƯƠNG 4: KHẢO SÁT VÀ KẾT QUẢ

4.1. Kết quả vận hành phần cứng của xe trên sa hình thực tế

Thực nghiệm vận hành trên hệ thống sa hình thực tế cho thấy cơ cấu phần cứng và thuật toán điều khiển phối hợp hoạt động khá ổn định. Trên các đoạn đường thẳng và đường cong, bộ điều khiển PID thể hiện tốc độ đáp ứng quá độ nhạy bén. Với các thông số được tinh chỉnh qua thực nghiệm và tốc độ chậm, khâu vi phân Kd đã làm tốt nhiệm vụ giảm độ vọt lố. Nhờ đó, xe bám vạch mượt mà, triệt tiêu được hiện tượng dao động lắc đuôi mạnh khi chuyển giao giữa đường cong và đường thẳng.

Khi sa hình trở nên phức tạp với các ngã rẽ vuông góc, việc nhận diện góc cua được thực hiện ổn. Hiện tượng xe văng nhẹ sau khi rẽ khiến các mắt cảm biến ngoài cùng chạm lại vào vạch đen, tạo ra tín hiệu rẽ giả mạo gửi lên máy tính làm sai lệch bản đồ đã được giải quyết nhờ thuật toán siết chặt điều kiện logic kết hợp với bộ đếm thời gian khóa ngã rẽ trong khoảng thời gian ms nhất định.

Kết quả vận hành của xe: [ket-qua-van-hanh-link](#)

4.2. Kết quả trực quan hóa bản đồ trên phần mềm máy tính

Phần mềm giám sát trên máy tính nhận các gói dữ liệu không đồng bộ từ vi điều khiển với định dạng cấu trúc đã quy ước. Dựa trên thời gian thực thi của lệnh đi thẳng và các mốc bẻ lái, thuật toán Python đã dựng lại thành công hình học của sa hình theo thời gian thực.

4.3. Sai số và Phân tích sai số

4.3.1. Các sai số thực nghiệm ghi nhận được trong quá trình lập bản đồ

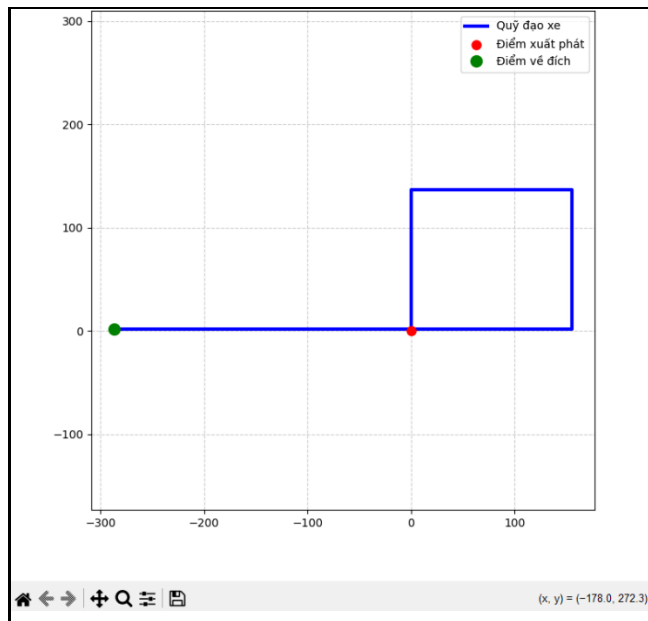
Trong quá trình vận hành đo đạc thực tế và đối chiếu với phần mềm vẽ bản đồ trên Python, nhóm em đã ghi nhận một số sai số hệ thống nhất định. Các vấn đề này chủ yếu biểu hiện trên giao diện trực quan hóa

Đầu tiên, nhóm em thấy có sai số về tỷ lệ chiều dài. Quãng đường đi thẳng hiển thị trên máy tính có sự sai lệch so với kích thước đo bằng thước dây dưới sa hình thực tế. Đặc biệt, ở các kịch bản chạy liên tục trong thời gian dài, chiều dài nét vẽ trên đồ họa có xu hướng bị co ngắn lại.

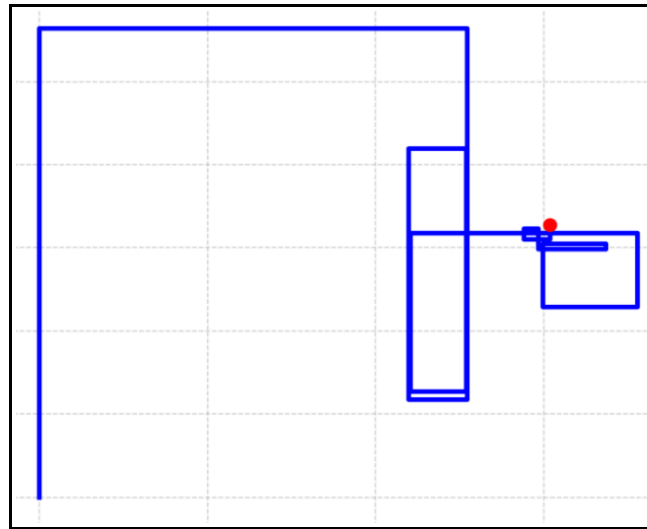
Bảng 5. Sai số giữa sa hình thực tế và sa hình dựng lại bằng phần mềm

Hành trình	Thực tế (cm)	Phần mềm(đơn vị)	Chênh lệch	Sai số (%)
Line 1	120	137	17	14.17
Line 2	117	162	45	38.46
Line 3	110	132	22	20
Line 4	80	106	26	32.5
Line 5	79	102	23	29.11
Line 6	65	84	19	29.23
Line 7	60	69	9	15
Line 8	36	45	9	25
Line 9	30	29	1	3.3
Tổng	697	866	171	24.4

Ngoài ra, nhóm cũng thấy có hiện tượng bỏ lỡ góc cua hoặc xuất hiện ngã rẽ ảo. Quá trình nhận diện ngã rẽ trên bản đồ bị phụ thuộc rất lớn vào thông số thời gian khóa góc cua. Nhóm ghi nhận hai trạng thái lỗi đối lập nhau. Khi cài đặt thời gian chờ nhận diện quá cao, bản đồ hiển thị xe sẽ không nhận các góc cua nằm quá sát nhau trên sa hình. Ngược lại, khi cài đặt thông số thời gian này quá thấp, trên bản đồ lại xuất hiện các nét vẽ ngã rẽ ảo do hiện tượng lác sau rẽ dù xe đang chạy trên đường thẳng.



Hình 8. Sơ hình thiếu các ngã rẽ do thiết lập thời gian chờ quá cao



Hình 9. Sơ hình quá nhiều ngã rẽ do thiết lập thời gian chờ quá ngắn

4.3.2. Phân tích nguyên nhân gây ra sai số

Hiện tượng sai lệch chiều dài xuất phát từ việc hệ thống không có cảm biến đo vận tốc để phản hồi vòng kín. Thuật toán vẽ bản đồ sử dụng phương pháp $S=V.t$, mặc định vận tốc V là hằng số. Tuy nhiên, khi vận hành liên tục, vận tốc thực tế của động cơ thay đổi. Sự mất đồng bộ giữa hằng số trên phần mềm và vận tốc vật lý thực tế đã gây ra sai số tích lũy về chiều dài trên đồ họa.

Về hiện tượng bỏ lỡ góc cua hoặc xuất hiện nhiều ngã rẽ ảo, nhóm em nhận định rằng đây là bài toán tối ưu. Khi thời gian khóa quá cao, hệ thống sinh ra trễ pha. Khóa chưa kịp mở khi xe đi tới các góc cua nằm quá sát nhau, khiến vi điều khiển bỏ qua tín hiệu cảm biến và lướt qua

ngã rẽ. Khi thời gian khóa quá thấp, hệ thống mất khả năng kháng nhiễu. Tần số lấy mẫu cao của vi điều khiển bắt trúng vùng dao động vọt lố tạo ra sau khi rẽ khiến hệ thống đọc nhầm thành ngã rẽ mới.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Qua quá trình nghiên cứu, thiết kế và thực nghiệm, nhóm em đã hoàn thành các mục tiêu cốt lõi đặt ra ban đầu.

Về mặt phần cứng và điều khiển, nhóm đã chế tạo và vận hành thành công mô hình xe tự hành dò line sử dụng vi điều khiển STM32. Thuật toán PID rời rạc được tinh chỉnh thành công. Hệ thống đã giải quyết được các bài toán logic cơ bản như nhận diện và xử lý góc cua, nhận diện vạch đích chéo, và tích hợp cơ chế bảo vệ khi xe có xu hướng trượt khỏi vạch.

Về mặt trực quan hóa dữ liệu, nhóm em cũng thiết lập thành công luồng truyền thông không đồng bộ (UART) giữa xe và máy tính. Đã xây dựng được phần mềm giao diện trên Python, có khả năng phân tích chuỗi dữ liệu thời gian thực để mô phỏng và tái tạo lại hình dáng hình học của sa hình dưới dạng bản đồ đồ họa.

5.2. Hướng phát triển

Mặc dù hệ thống đã hoạt động ổn định và đáp ứng được các kịch bản sa hình cơ bản, giải pháp ước lượng như hiện tại vẫn tồn tại sai số tích lũy. Để nâng cấp hệ thống tiến gần hơn đến các chuẩn mực công nghiệp trong lĩnh vực Robot di động nhóm em cần phải có các hướng phát triển tiếp theo.

Để khắc phục triệt để các sai số bản đồ, hệ thống cần được tích hợp các cảm biến khác như Encoder và module quán tính IMU. Encoder giúp thiết lập bộ điều khiển PID vòng kín để tính toán quãng đường tịnh tiến dựa trên số xung vật lý thay vì thời gian, loại bỏ sai số. Đồng thời, việc trích xuất góc xoay tuyệt đối từ IMU sẽ cung cấp dữ liệu định hướng không gian thực. Nhờ đó, nhóm em có thể vẽ chính xác quỹ đạo bề lái thực tế thay vì giả định góc 90° , giải quyết sai lệch đồ họa do bánh xe bị trượt cơ khí tại các góc cua gắt.

TÀI LIỆU THAM KHẢO

- [1] STMicroelectronics, *RM0368 Reference manual: STM32F401xB/C and STM32F401xD/E advanced ARM®-based 32-bit MCUs*.
- [2] Toshiba Electronic Devices & Storage Corporation, *TB6612FNG Datasheet*
- [3] Vishay Semiconductors, *TCRT5000 / TCRT5000L Reflective Optical Sensor Datasheet*.
- [4] Shenzhen Jiayuan Electronic Co., *JDY-31 Bluetooth SPP Module Datasheet*.